

# FROST Guide

Threshold custody for randomised RedPallas keys

m@rek.onl

## Abstract

This volume of *The Zcash Arboretum* constructs threshold spend authorisation: several holders of key shares produce one ordinary RedPallas signature without reconstructing the spending key during signing. It assumes interpolation from the *Math Guide*, Schnorr and RedPallas from the *Crypto Guide*, Action authorisation from the *Ironwood Guide*, and separated transaction-building roles from the *Wallet Guide*. The construction includes share provisioning, transcript encodings, nonce state, randomisation and the final signing workflow. Algebraic correctness, imported unforgeability results and unproved composition requirements are distinguished explicitly.

# Contents

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Threshold custody and its contract</b>              | <b>3</b>  |
| 1.1      | The selected group, encodings, and hashes . . . . .    | 3         |
| 1.2      | Public and secret objects . . . . .                    | 3         |
| <b>2</b> | <b>Shares and verifiable key generation</b>            | <b>4</b>  |
| 2.1      | A polynomial with the key at zero . . . . .            | 4         |
| 2.2      | Checking a dealer's shares . . . . .                   | 5         |
| 2.3      | Generating shares without a single dealer . . . . .    | 5         |
| 2.4      | Normalising an Orchard authorisation key . . . . .     | 6         |
| <b>3</b> | <b>Two-round signing</b>                               | <b>6</b>  |
| 3.1      | Nonce generation and first-round commitments . . . . . | 6         |
| 3.2      | The canonical signing package . . . . .                | 7         |
| 3.3      | Responses and aggregation . . . . .                    | 7         |
| 3.4      | A scalar-field walkthrough . . . . .                   | 8         |
| <b>4</b> | <b>Security and nonce state</b>                        | <b>8</b>  |
| 4.1      | What the binding factor prevents . . . . .             | 8         |
| 4.2      | The imported signing guarantee . . . . .               | 9         |
| 4.3      | Why a nonce pair has only one response . . . . .       | 9         |
| <b>5</b> | <b>Randomised RedPallas signing</b>                    | <b>10</b> |
| 5.1      | Shifting all shares . . . . .                          | 10        |
| 5.2      | The selected randomizer construction . . . . .         | 10        |
| 5.3      | Signature compatibility . . . . .                      | 11        |
| 5.4      | Privacy and the remaining theorem boundary . . . . .   | 11        |
| <b>6</b> | <b>Transaction construction with separated signers</b> | <b>11</b> |
| 6.1      | The order that avoids a circular digest . . . . .      | 11        |
| 6.2      | Applying the shared payment example . . . . .          | 12        |
| 6.3      | What a completed endpoint guarantees . . . . .         | 12        |

# 1 Threshold custody and its contract

*Math Guide*, §Groups and scalar multiplication and §Prime fields and modular arithmetic, supplies the group and scalar language used here. *Crypto Guide*, §Security contracts and reductions, defines signature unforgeability and the adversarial experiments used to state it.

A spending key normally belongs to one signer. Threshold custody replaces that signer by  $n$  participants with a threshold  $t$ : any authorised set of at least  $t$  participants can cooperate, while fewer than  $t$  should not be able to forge a signature. The chain still sees one public key and one signature. It does not learn the threshold or the selected participants from an additional on-chain list of signatures.

This volume selects the signing mechanism of RFC 9591, Sections 4–5, and its Pallas integration in frost at 2016e44b and reddsa at 3792daa9. The exact randomizer construction is the seed-and-commitments API in frost-rerandomized/src/lib.rs; the group and hash functions are in reddsa/src/frost/redpallas.rs. ZIP 312 supplies the Zcash compatibility requirements, but its direct-randomizer helper is not silently substituted for this selected implementation.

The endpoint is threshold authorisation of an Orchard-style Action. No Crosslink, voting or future recovery protocol is needed to understand that endpoint.

## 1.1 The selected group, encodings, and hashes

A *ciphersuite* is a fixed choice of group, encodings, and hash functions. *Crypto Guide*, §Hash functions and key derivation, constructs the BLAKE2b operation used in this suite. We fix the functions before using them in the protocol equations.

The group is Pallas, with order  $q = p_{\text{Vesta}}$  and base  $G = G^{\text{SpendAuth}}$ , as constructed in *Crypto Guide*, §RedPallas signatures. Canonical compressed points occupy 32 bytes. Scalar encoding is a 32-byte little-endian integer strictly less than  $q$ ; decoding rejects an out-of-range integer rather than reducing it. Participant identifiers additionally reject zero.

The group decoder rejects malformed points and the identity at FROST input positions that require non-identity elements. Algebraic identities below still hold when a group sum is the identity, but the concrete ciphersuite’s serialisation and validation errors govern whether that rare session can be completed. It must fail cleanly, not reinterpret an invalid encoding.

For a 64-byte string  $b$ , write  $\text{Wide}(b) = \text{LEInteger}(b) \bmod q$ . The scalar-valued hashes apply this map to BLAKE2b-512’s output. The complete personalised functions are:

| Function | Output   | BLAKE2b personalisation |
|----------|----------|-------------------------|
| $H_1$    | scalar   | FROST_RedPallasR        |
| $H_2$    | scalar   | Zcash_RedPallasH        |
| $H_3$    | scalar   | FROST_RedPallasN        |
| $H_4$    | 64 bytes | FROST_RedPallasM        |
| $H_5$    | 64 bytes | FROST_RedPallasC        |
| $H_R$    | scalar   | FROST_RedPallasA        |
| $H_D$    | scalar   | FROST_RedPallasD        |

The hash  $H_D$  supports participants jointly generating shares of a key without giving any participant the complete secret. It is an additional library function, not an RFC signing hash. Identifiers may simply be assigned as distinct nonzero scalars; no identifier hash is needed for that choice. We write  $P^*$  for the canonical encoding of a point  $P$ , and  $\text{enc}(x)$  for a scalar or identifier encoding. The selected key-generation API takes 16-bit participant counts and requires  $2 \leq t \leq n$ , although the mathematical sharing construction also makes sense at  $t = 1$ .

## 1.2 Public and secret objects

Work in an additive prime-order group  $\mathbb{G} = \langle G \rangle$  of order  $q$ . Scalars belong to  $\mathbb{F}_q$ . The Pallas instantiation will set  $q = p_{\text{Vesta}}$ . The *Crypto Guide*, §Knowledge, simulation, and Schnorr, constructs the

verification equation

$$[z]G = R + [c]Y, \quad c = H_2(R^* || Y^* || m).$$

Here  $Y = [x]G$  is the group public key,  $m$  is the approved byte-string message,  $R$  is the nonce commitment and  $z$  the response scalar. FROST distributes the calculation of  $z$ ; it does not change this outside verification rule.

Each participant has a distinct nonzero identifier  $i \in \mathbb{F}_q$ , a secret signing share  $x_i$ , and a public verification share  $Y_i = [x_i]G$ . A public key package fixes  $Y$ , all identifiers and their  $Y_i$ , and the threshold. A signer does not accept a coordinator's replacement verification-share map as a new definition of its key.

The coordinator collects nonce commitments, distributes one signing package and aggregates response shares. Its ability to schedule or interrupt the protocol does not give it spending authority. Conversely, unforgeability does not oblige a malicious coordinator or selected signer to finish a session.

**Definition 1.1** (Threshold authorisation requirement). Fix an honestly provisioned key and an adversary controlling fewer than  $t$  participants. Honest participants approve the complete application message before answering a signing request. A successful forgery is an accepting signature outside the signing interactions permitted by the security game's freshness rule: the rule specifying which earlier signing interactions disqualify a proposed forgery. The game must state when participants may be corrupted, whether sessions may overlap, who chooses each key randomizer, and the exact disqualifying interactions.

These qualifications matter. A new signature on a previously signed message is excluded by an ordinary new-message game but included by a strong-unforgeability game. A theorem for corruption choices fixed before key generation does not automatically cover secrets exposed adaptively during signing. We identify the imported guarantees in §4.

## 2 Shares and verifiable key generation

The interpolation theorem is proved in *Math Guide*, §*Polynomials and evaluation*. We use it to construct a secret sharing and then to explain precisely what the public share checks add. That same section proves the hidden-constant-coefficient lemma used for the privacy statement below.

### 2.1 A polynomial with the key at zero

Choose  $1 \leq t \leq n < q$ , a secret  $x \in \mathbb{F}_q$ , and independent uniform coefficients  $a_1, \dots, a_{t-1} \in \mathbb{F}_q$ . Define

$$f(X) = x + \sum_{j=1}^{t-1} a_j X^j, \quad x_i = f(i).$$

The degree is at most  $t - 1$ ; a highest coefficient may be zero. Identifiers cannot be zero because  $f(0)$  is the secret itself.

For a signing set  $S$  of distinct identifiers with  $|S| \geq t$ , define

$$\lambda_i^S = \prod_{\substack{j \in S \\ j \neq i}} \frac{j}{j - i}.$$

All denominators are invertible because identifiers are distinct. Evaluation of the interpolation formula at zero gives

$$\sum_{i \in S} \lambda_i^S x_i = x, \quad \sum_{i \in S} \lambda_i^S = 1.$$

The second equality is interpolation of the constant polynomial 1. It will be essential when all shares receive the same randomizer.

This construction is called *Shamir secret sharing*. Any fixed set of  $r < t$  private shares is jointly uniform in  $\mathbb{F}_q^r$ , independently of the fixed secret  $x$ : apply the lemma “A hidden constant coefficient” in *Math Guide*, §*Polynomials and evaluation*, with its constant  $s = x$  and evaluation points equal to the participant identifiers. Its requirements are exactly the nonzero distinct identifiers and independent uniform nonconstant coefficients enforced here.

Publishing  $Y = [x]G$  changes the information-theoretic statement: an unbounded observer can recover  $x$  from  $Y$ . Threshold key secrecy with public verification data is computational. The private-share statement must not be quoted as perfect privacy of the entire public-key protocol.

## 2.2 Checking a dealer’s shares

A trusted dealer can distribute the shares over confidential, authenticated channels and publish coefficient commitments

$$C_j = [a_j]G \quad (0 \leq j < t), \quad a_0 = x.$$

Each recipient checks

$$[x_i]G = \sum_{j=0}^{t-1} [i^j]C_j.$$

The right side equals  $[f(i)]G$ . Since scalar multiplication by a generator is injective over  $\mathbb{F}_q$ , the equality verifies that the received scalar is the evaluation described by those public commitments.

All participants must receive the same commitment list. Otherwise a dealer could give individually consistent but mutually incompatible views. An authenticated private channel identifies the sender to one recipient; it does not by itself establish agreement between recipients.

The dealer knows  $x$ . Splitting an existing key is therefore compatible with a seed-derived account, but its threshold guarantee requires the original full key and dealer randomness to be erased or otherwise accounted for. Dividing a key does not erase previously made backups.

## 2.3 Generating shares without a single dealer

The selected library also implements distributed key generation (DKG) in `frost-core/src/keys/dkg.rs`. It runs a dealer contribution from every participant and adds the results. We state its essential messages, including the checks needed to obtain one common key.

Each participant  $\ell$  samples a polynomial  $f_\ell(X) = \sum_{j=0}^{t-1} a_{\ell j} X^j$ . In the selected implementation the constant coefficient is sampled nonzero, as a signing key; the other coefficients are independent uniform scalars. It publishes  $C_{\ell j} = [a_{\ell j}]G$ . It proves knowledge of its constant coefficient using a Schnorr proof, with a fresh nonzero nonce  $u_\ell$ :

$$U_\ell = [u_\ell]G, \quad e_\ell = H_D(\text{enc}(\ell) \| C_{\ell 0}^* \| U_\ell^*), \quad v_\ell = u_\ell + e_\ell a_{\ell 0}.$$

The verifier recomputes  $e_\ell$  and checks  $[v_\ell]G = U_\ell + [e_\ell]C_{\ell 0}$ . The selected implementation hashes exactly these three fields. Its hash function separates the DKG role; an identifier for this one key-generation run, called a ceremony identifier, and membership agreement remain responsibilities of the enclosing transport protocol, not extra inputs we invent for this library call.

After agreeing on all first-round packages, participant  $\ell$  sends  $f_\ell(i)$  privately to recipient  $i$ . Each recipient verifies

$$[f_\ell(i)]G = \sum_{j=0}^{t-1} [i^j]C_{\ell j}$$

for every contributor. It rejects missing, duplicate, unauthenticated or inconsistent packages. On success it forms

$$x_i = \sum_{\ell=1}^n f_{\ell}(i), \quad Y_i = \sum_{\ell=1}^n \sum_{j=0}^{t-1} [i^j] C_{\ell j}, \quad Y = \sum_{\ell=1}^n C_{\ell 0}.$$

The sum  $F(X) = \sum_{\ell} f_{\ell}(X)$  has degree at most  $t-1$ ,  $x_i = F(i)$ , and  $Y = [F(0)]G$ . Thus the shares have exactly the interpolation relation signing needs.

The private contributions and polynomial coefficients are erased after their required checks and aggregation. The enduring private object is the participant's resulting key share. The enduring public object is the agreed key package.

This is an aborting DKG, not a proof that malicious participants cannot block key generation. A proof of knowledge of each contribution also does not by itself prove unbiased output when an adversary aborts unfavourable runs and allows only favourable runs to complete. The signing theorem below does not silently acquire a DKG-composition guarantee merely because the final interpolation identities are correct.

## 2.4 Normalising an Orchard authorisation key

Orchard's long-term spend-authorisation key uses the even-y representative; *Ironwood Guide*, §*Keys and viewing capabilities*, defines the convention. If the generated  $Y$  has odd  $y$ , negate every secret share and corresponding public share, the group key, and any retained polynomial commitments:

$$x'_i = -x_i, \quad Y'_i = -Y_i, \quad Y' = -Y.$$

Then  $\sum_i \lambda_i^S x'_i = -x$ , so the new shares interpolate to the normalised group key. Negating only the group key would leave an inconsistent key package.

The selected RedPallas `post_generate` and `post_dkg` hooks apply this transformation. The later per-Action randomised key has no even-y requirement. Applying the normalisation again to that key would change the transaction's authorisation equation.

## 3 Two-round signing

We first construct signing for one fixed group key  $Y$ . The later randomisation step changes the key inputs but preserves the equations.

### 3.1 Nonce generation and first-round commitments

For each signing session, participant  $i$  creates two nonce scalars  $d_i, e_i$ , called the hiding and binding nonces. The RFC's hedged nonce generator combines fresh randomness with the secret share:

$$\text{Nonce}(x_i; b) = H_3(b \parallel \text{enc}(x_i)), \quad b \leftarrow \{0, 1\}^{256}.$$

Invoke it independently twice, using two fresh 32-byte strings, to obtain  $d_i, e_i$ . The participant retains the scalars and sends only

$$D_i = [d_i]G, \quad E_i = [e_i]G$$

to the coordinator. Hashing the secret share into nonce generation does not make repeated random inputs safe. Identical inputs repeat the nonce.

The participant associates the pair with durable single-use state. Publishing a commitment is not permission to reuse its secret in several responses. Preprocessing may prepare pairs before the message is known; each pair is still consumed by at most one signing operation.

### 3.2 The canonical signing package

The coordinator selects  $S$ , with  $t \leq |S| \leq n$ , and constructs

$$B = ((i, D_i, E_i))_{i \in S}$$

in ascending numerical identifier order. The group encoding is

$$\text{Enc}(B) = \prod_{i \in S} (\text{enc}(i) \| D_i^* \| E_i^*).$$

Within one ciphersuite all three encodings have fixed lengths; this makes the concatenation unambiguous. Ascending scalar order is not lexicographic order of little-endian byte strings: that order compares the first differing byte, not the integers represented by all bytes.

Each signer receives  $m, B$ , validates every point and identifier, checks uniqueness and ordering, checks that the selected participants belong to the stored key package, and verifies that its own identifier and exact commitments occur once. It must also approve  $m$  according to the application; a syntactically valid byte string is not necessarily an authorised payment.

Define the message and commitment-list prehashes and binding factors:

$$\begin{aligned} M &= H_4(m), & C &= H_5(\text{Enc}(B)), \\ \rho_i &= H_1(Y^* \| M \| C \| \text{enc}(i)). \end{aligned}$$

These are RFC 9591's ordered inputs. The public key, message, complete commitment list and identifier all participate. The symbol  $\rho_i$  here denotes a FROST binding factor, not a note's uniqueness seed.

Each participant computes

$$R_i = D_i + [\rho_i]E_i, \quad R = \sum_{i \in S} R_i, \quad c = H_2(R^* \| Y^* \| m).$$

No secret nonce is needed to recompute these public values.

### 3.3 Responses and aggregation

Participant  $i$  calculates the Lagrange coefficient for this exact  $S$ , and returns

$$z_i = d_i + \rho_i e_i + c \lambda_i^S x_i.$$

The coordinator checks scalar encodings, the response-to-identifier mapping and completeness of the selected response set. It computes

$$z = \sum_{i \in S} z_i, \quad \sigma = R^* \| \text{enc}(z).$$

The public verification equation is the ordinary Schnorr equation. A coordinator may first verify the aggregate and verify individual shares only after aggregate failure. To check a share, it tests

$$[z_i]G = R_i + [c \lambda_i^S]Y_i.$$

These checks use the stored  $Y_i$ , not a public share supplied alongside an allegedly bad response. Authenticated messages make a failed check attributable to its sender; the equation alone does not prove who sent an arbitrary byte string.

**Theorem 3.1** (Signing correctness). *If all selected shares belong to the same key polynomial, all participants use the same valid signing package, and all computed values satisfy the ciphersuite’s encoding and nonidentity checks, the aggregate verifies under  $Y$ . Freshness of the nonce pairs is additionally required for security, not for this algebraic identity.*

*Proof.* Summing the response equations and using interpolation gives

$$z = \sum_{i \in S} (d_i + \rho_i e_i) + c \sum_{i \in S} \lambda_i^S x_i = r + cx, \quad r = \sum_{i \in S} (d_i + \rho_i e_i).$$

The group commitment is  $R = [r]G$ . Multiplying the scalar equality by  $G$  yields  $[z]G = R + [c]Y$ .  $\square$

A signer may leave before answering. The coordinator cannot simply drop it and sum the others: the participant set is inside the binding factors and interpolation weights, and the dropped commitments are inside  $R$ . A retry selects fresh nonce pairs and constructs a fresh signing package.

### 3.4 A scalar-field walkthrough

For an arithmetic illustration only, take  $q = 101$ ,  $f(X) = 42 + 17X + 29X^2$ , and  $S = \{1, 3, 5\}$ . The shares and coefficients are

$$(x_1, x_3, x_5) = (88, 51, 44), \quad (\lambda_1, \lambda_3, \lambda_5) = (65, 24, 13).$$

Both  $\sum_i \lambda_i = 1$  and  $\sum_i \lambda_i x_i = 42$  hold modulo 101. Choose illustrative nonce pairs

$$(d_1, d_3, d_5) = (11, 13, 17), \quad (e_1, e_3, e_5) = (19, 23, 29).$$

For illustrative hash outputs  $(\rho_1, \rho_3, \rho_5) = (2, 4, 6)$  and  $c = 7$ , the effective nonces are  $(49, 4, 90)$ , whose sum is 42. The response shares are  $(93, 88, 54)$ ; their sum is 33. Finally,

$$33 = 42 + 7 \cdot 42 \pmod{101}.$$

The retained numerical check recomputes every entry. These invented hash outputs and small scalars test the algebra; they are not a ciphersuite test vector or an instance with cryptographic security.

## 4 Security and nonce state

Correctness says honest responses add to a valid signature. Security must explain why a malicious coordinator interacting with several sessions cannot obtain an unapproved one. The equations alone do not establish that conclusion. The random-oracle model, which treats each distinct hash input as receiving one independently random answer consistently reused on repetition, is constructed in *Crypto Guide*, §*Security contracts and reductions*.

### 4.1 What the binding factor prevents

A response is linear in the secret quantities:

$$z_i = d_i + \rho_i e_i + c \lambda_i^S x_i.$$

If the coordinator could select the coefficients of this equation independently after collecting nonce commitments, it could try to combine responses from separate sessions into an equation it wanted. The binding factor makes the effective nonce depend on the entire signing package, including the eventual message and public key. Changing another participant’s commitment therefore changes the honest participant’s coefficient too.

The domain-separated hashes are essential here. Calling two different functions the same random oracle, or omitting the participant list from one side’s calculation, changes the protocol analysed by the security proof. The fact that the resulting aggregate still resembles a Schnorr signature is not a repair.



## 4.2 The imported signing guarantee

The signing-layer result we rely on is the analysis of FROST1 in Bellare, Crites, Komlo, Maller, Tessaro and Zhu, *Better than Advertised Security for Non-interactive Threshold Signatures*, CRYPTO 2022; the FROST analysis is retained in ePrint 2022/833, Theorem 5.4. Its setting uses random oracles, trusted-dealer key distribution and static corruption, meaning the corrupted set is fixed before key generation, of fewer than  $t$  participants. Its strong threshold-forgery notion also accounts for which participants contributed to a session; an ordinary single-signer theorem against new-message forgery after chosen signing queries is not that result.

The hardness assumption is *one-more discrete logarithm*. In its game, an adversary obtains independent random challenge group elements and may ask a discrete-log oracle for selected logarithms. It must not be able to output the logarithms of more challenge elements than the number of oracle queries it used, with non-negligible probability. This assumption permits oracle access that the ordinary discrete-log game does not. We do not equate the two assumptions.

The imported result covers the matching signing construction under its stated key-generation and corruption hypotheses. It does not prove that the implemented DKG composes with signing, that arbitrary adaptive corruptions are safe, or that backup software preserves the nonce requirements. Those are separate obligations. The purpose of importing a theorem is to state a precise guarantee, not to replace the unforgeability question with an adjective.

## 4.3 Why a nonce pair has only one response

If one nonce pair is answered under three packages, the attacker obtains equations

$$z_j = d + \rho_j e + \beta_j x_i, \quad \beta_j = c_j \lambda_i^{s_j}, \quad j = 1, 2, 3.$$

Subtract the first equation from the other two. This leaves two linear equations in  $e, x_i$ , solved by the elimination procedure in *Math Guide*, §*Linear algebra and uniform masks*. Whenever

$$(\rho_2 - \rho_1)(\beta_3 - \beta_1) - (\rho_3 - \rho_1)(\beta_2 - \beta_1) \neq 0,$$

the coefficient matrix is invertible and reveals  $x_i$ , then  $d$ . Fewer observations can suffice in special cases or with extra leaked information. A failed aggregate does not make an emitted response harmless: the attacker still possesses its linear equation.

Nonce generation from a deterministic function of just the share and message is therefore unsuitable. Signing the same message in another coordinator-chosen package would repeat the secret pair while changing its public coefficients. Fresh randomness and single-use storage solve different parts of this problem; neither substitutes for the other.

**Construction 4.1** (Application nonce state). Associate each generated pair with a unique local record.

1. Store the private pair as available before advertising its public commitments.
2. Bind that record to one complete accepted signing request.
3. Irreversibly mark it consumed before releasing any response derived from it. A crash must not roll that transition back.
4. Erase the private pair when its response is computed or the session is abandoned. A restart obtains new commitments.

This state machine is an application requirement, not a persistence service furnished by a scalar-signing function. The selected Rust API takes references to nonce objects; the surrounding application must not call it again with a consumed pair. Restoring a backup, cloning a device or racing two

signing requests must not duplicate available nonce state. Ordinary durable key backups should not restore an old pool of preprocessed nonces.

A malicious participant can withhold a response. Share verification can identify inconsistent responses from authenticated senders; it cannot force participation or prove that a sender intended to disrupt the session. Availability policy and retry policy remain outside the unforgeability theorem.

## 5 Randomised RedPallas signing

The *Ironwood Guide*, §*Balance, digests, and authorisation*, publishes a per-Action key

$$\text{rk} = Y + [\alpha]G$$

rather than the long-term  $Y = \text{ak}$ . The Action proof establishes the relation to the authorised spend key. The outside signature must verify under  $\text{rk}$ , not under  $Y$ .

### 5.1 Shifting all shares

For a common scalar  $\alpha$ , define

$$\bar{x}_i = x_i + \alpha, \quad \bar{Y}_i = Y_i + [\alpha]G, \quad \bar{Y} = Y + [\alpha]G.$$

All signers use  $\bar{Y}$  in the binding factors and signature challenge and  $\bar{x}_i$  in their response equation. The coordinator uses  $\bar{Y}_i$  for share verification and  $\bar{Y}$  for final verification.

**Proposition 5.1** (Randomisation commutes with sharing). *The shifted shares interpolate to  $x + \alpha$  for every permitted signing set  $S$ . Their FROST aggregate is an ordinary signature under  $\bar{Y}$ .*

*Proof.* The two interpolation identities give

$$\sum_{i \in S} \lambda_i^S (x_i + \alpha) = x + \alpha \sum_{i \in S} \lambda_i^S = x + \alpha.$$

Equivalently, the shifted polynomial is  $f(X) + \alpha$ , whose degree remains at most  $t - 1$ . Theorem 3.1 then applies to the shifted inputs.  $\square$

Adding the same scalar to every share does not add  $|S|\alpha$  to the group key, because shares are interpolated rather than simply summed. This is why no separate sharing ceremony for the randomizer is needed.

### 5.2 The selected randomizer construction

Let  $B$  be the exact first-round commitment list for the Action. The coordinator draws a 32-byte seed  $s$  and computes

$$\alpha = H_R(s \parallel \text{Enc}(B)).$$

It sends  $s, B$  to each participant. Each participant regenerates  $\alpha$  locally and constructs the shifted key package.

This is the mechanism of `Randomizer::new_from_commitments` and `regenerate_from_seed_and_commitments` at the selected frost commit. The message is *not* inside this randomizer hash. It is inside  $H_4$ , every binding factor and the signature challenge. That distinction permits the builder to calculate  $\text{rk}$  before the transaction digest containing  $\text{rk}$  is known.

Fresh commitments contribute input variation even if the coordinator's seed generator malfunctions. This is hedging, not a theorem that every maliciously selected seed produces an unbiased randomizer. A coordinator can examine candidate seeds and can learn  $\alpha$ . Its privacy role must be accounted for explicitly.

### 5.3 Signature compatibility

Every list entry is therefore  $32 + 32 + 32 = 96$  bytes before transport framing. The final signature is  $32 + 32 = 64$  bytes. Neither number depends on whether the selected signing set has two participants or twenty; only the intermediate transcript grows with its size.

The essential compatibility condition is  $H_2$ . Its personalisation, input order, scalar conversion and point/scalar encodings are exactly RedPallas's. A verifier does not care whether the valid response was computed from one secret or from interpolated shares. Other FROST ciphersuites are not interchangeable merely because they also output 64-byte signatures.

### 5.4 Privacy and the remaining theorem boundary

For an independent uniform  $\alpha$ , the map  $\alpha \mapsto Y + [\alpha]G$  is a bijection from  $\mathbb{F}_q$  to  $\mathbb{G}$ . Thus the randomised public key alone is uniform, independently of  $Y$ . With wide reduction of a random-oracle output, the distribution is statistically close to uniform, rather than exactly uniform.

A person learning  $\alpha$  can compute  $Y = rk - [\alpha]G$ . The coordinator, signers and transaction prover therefore are not covered by a claim that their own signing sessions are unlinkable to the long-term key. The seed must travel over a confidential, authenticated channel:  $s$  and public  $B$  reveal  $\alpha$ . Transaction timing, signer identity and network traffic may reveal additional links.

Gouvêa and Komlo's *Re-Randomized FROST* provides the algebraic construction and a claimed unforgeability analysis. Its stated freshness condition concerns new messages; it does not establish the stronger fresh-message-signature-pair property requested by the Zcash specification. The selected seed-and-commitments API also needs its precise correspondence to an analysed randomizer model. We do not present the preprint's claimed concrete bound as a proved bound for this full implementation and DKG composition.

The identity in Proposition 5.1, consensus verification compatibility and the public-key masking calculation are proved here. A full threshold strong-unforgeability and joint privacy theorem for the complete selected workflow remains a separate security obligation.

## 6 Transaction construction with separated signers

A signer receiving only a 32-byte digest cannot determine what payment it authorises. Threshold cryptography does not fix a missing transaction review. The complete signer contract is the one in *Wallet Guide*, §*Constructing and authorising a transaction*; this section inserts the threshold rounds at the necessary points.

### 6.1 The order that avoids a circular digest

For each real spend assigned to a threshold key:

1. Select the participants and obtain their unused first-round commitments. At this point the final transaction message need not exist.
2. Freeze  $B$ , generate  $s$ , derive  $\alpha$ , and construct  $rk = Y + [\alpha]G$ .
3. Insert that  $rk$  into the Action. Complete the transaction effects, including outputs, public balances, fee and all other fields covered by the applicable signature digest.
4. Compute the transaction's signature digest  $m$ . Give each signer the transaction information needed to verify its authorisation policy, along with  $m, B, s$  and the identification of the Action it is authorising.
5. Each signer checks that its regenerated  $rk$  equals the Action's key, recomputes the required digest, approves the transaction, and produces one response using its consumed nonce pair.

6. Aggregate, verify under the Action's  $rk$ , and insert the resulting spend-authorisation signature. Complete the proof and remaining authorising data in the order their own interfaces permit.

The proof relation uses the same  $\alpha$  that defines the signature key. The prover must possess the required proof-authorisation data; threshold spend-key shares alone are not the entire Orchard witness. Likewise, withholding the spend key from the prover does not make a malicious builder's proposed outputs safe to approve.

The final signature digest may exclude some authorising data by design. The signer follows the exact digest definition for that transaction version; it must not hash an arbitrary serialisation or replace the digest by the transaction identifier. A partially created transaction container transports roles and fields; it is not itself proof that the signer checked them.

## 6.2 Applying the shared payment example

The *Wallet Guide*'s running transaction spends an 80,000-zatoshi note, pays 50,000, returns 20,000 in change and pays a 10,000 fee. Its two padded Actions may have net values +30,000 and -20,000. FROST changes none of that arithmetic.

The holders of the real note's threshold spend authority jointly sign its Action. A dummy spend still follows its own permitted authorisation construction; it does not represent another threshold-owned 80,000 zatoshi note. Before approving, the real signers check recipient, amount, change ownership, fee and the relevant Action/digest bindings. The aggregate signature does not reveal which subset approved, but the participants and coordinator know their own session.

The binding signature is a different role. It supplies the authorisation component of the balance argument, whose opening-knowledge premises, proof relations and integer bounds are stated in *Ironwood Guide*, §*Balance, digests, and authorisation*. Threshold spend authorisation does not replace that proof of balance or distribute its key automatically.

## 6.3 What a completed endpoint guarantees

A complete honest run outputs the same signature format the existing Action verifier accepts, with no reconstruction of the spend key during signing. The off-chain system must still provide an agreed key package, participant authentication, private share distribution, confidential randomizer transport, application-level transaction approval and durable nonce consumption.

Those requirements are not optional implementation polish: each closes a specific step in the construction. The threshold-signing endpoint is complete only when the surrounding custody system implements them. Nothing in this construction turns a proposed threshold application for some other protocol into an implemented one.